Log Analysis: Safe Regex Practices & Catastrophic Backtracking Prevention

Duration : 45 Min.

Scenario

A log extraction regex causes high CPU usage and hangs due to catastrophic backtracking (ReDoS). Refactor the regular expression to be safe.

Target File Location & Creation

File to Create/Update: `student_workspace/solution.json` **Input Resource File to Inspect**: `student_workspace/redos_audit.log`

Task Objectives

Perform the following actions inside the `student_workspace` directory: - Inspect `redos_audit.log` in `student_workspace/`. - Edit `solution.json` and record the required log analytics or findings.

Instructions to Perform the Task

1. When your workspace loads in **VS Code**, use the **Explorer** panel on the left to locate your files.
2. Open and inspect `redos_audit.log` inside `student_workspace/`.
3. Open `solution.json` in `student_workspace/` and perform the required modifications.
4. Save your changes (`Ctrl + S` or `Cmd + S`).
5. Open the built-in terminal by clicking **Terminal > New Terminal** from the top menu.
6. Verify your progress by running `python run.py` locally in the terminal.

Validation

Once you have saved your files and verified your progress, return to the platform dashboard and click the **"Run Test" / "Verify"** button. This will automatically evaluate your changes and generate your score!

Grading Criteria

Your performance will be evaluated based on the following test cases:

Test Case	Requirement	Marks
TC1	safe_regex.json exists	3 Marks

TC2	Nested quantifier patterns (e.g. (a+)+) removed	3 Marks
TC3	Non-greedy or atomic group match syntax applied	2 Marks
TC4	Regex execution completes under 10ms	2 Marks

Total Score: 10 Marks

Important Notes

- This is an auto-evaluated question. Ensure all code edits are properly saved and the 'run.py' checks pass before submission.